

Improving User Engagement and Learning Outcomes in LLM-Based Python Tutor: A Study of PACE

Muhtasim Ibtada Shochcho*
Center for Computational & Data
Sciences

Independent University, Bangladesh
Dhaka, Bangladesh
2021111@iub.edu.bd

Mohammad Ashfaq Ur
Rahman*

Center for Computational & Data
Sciences
Independent University, Bangladesh
Dhaka, Bangladesh
imashfaqfardin@gmail.com

Shadman Rohan

Center for Computational & Data
Sciences
Independent University, Bangladesh
Dhaka, Bangladesh
shadman@iub.edu.bd

Ashraful Islam

Center for Computational & Data
Sciences
Independent University, Bangladesh
Dhaka, Bangladesh
ashraful@iub.edu.bd

Hasnain Heickal

University of Massachusetts Amherst
Amherst, Massachusetts, USA
hheickal@cs.umass.edu

AKM Mahbubur Rahman

Center for Computational & Data
Sciences
Independent University, Bangladesh
Dhaka, Bangladesh
akmmrahman@iub.edu.bd

M. Ashraful Amin

Center for Computational & Data
Sciences
Independent University, Bangladesh
Dhaka, Bangladesh
aminmdashraful@iub.edu.bd

Amin Ahsan Ali

Center for Computational & Data
Sciences
Independent University, Bangladesh
Dhaka, Bangladesh
aminali@iub.edu.bd

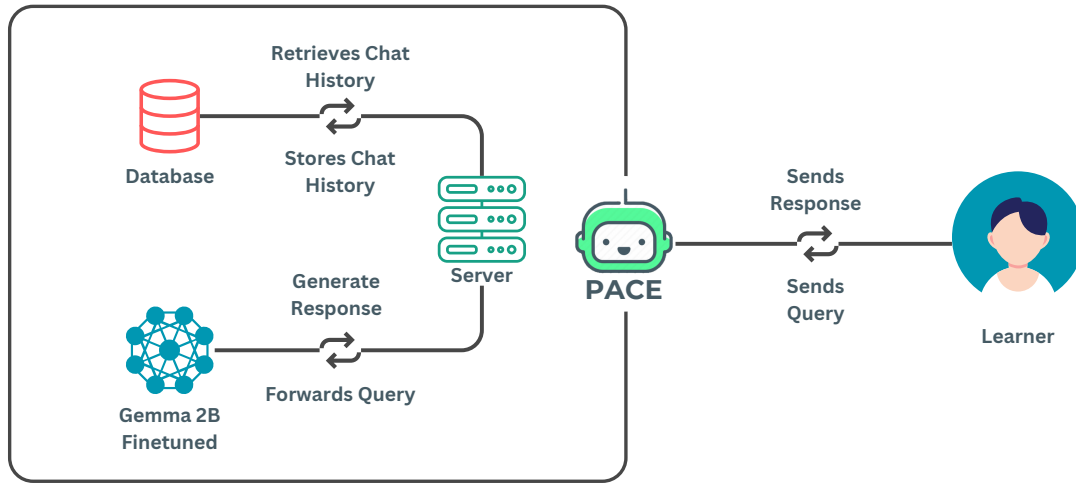


Figure 1: The diagram illustrates the flow between the database, chat server, and the PACE conversational model to retrieve chat history and generate personalized responses for student evaluation and engagement

*Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '25, Yokohama, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1395-8/25/04

<https://doi.org/10.1145/3706599.3720240>

Abstract

Large Language Models (LLMs) are increasingly being adopted for educational applications, but sometimes, limited internet access and budget constraints restrict their accessibility. Small Language Models (SLMs) have emerged as viable alternatives, capable of providing effective tutoring in resource-constrained contexts. This paper introduces PACE (Python AI Companion for Enhanced Engagement), a system leveraging SLMs to deliver step-by-step guidance and

adaptive feedback for teaching Python. An evaluation with varying levels of learners showed PACE's effectiveness, achieving a System Usability Scale (SUS) score of 77.28. While participants were generally satisfied with its clarity and personalized feedback, they identified some areas for improvement, such as loss of context during lengthy conversations. This study examines (1) the PACE system's effectiveness in programming education according to learners, (2) learners' trust in PACE versus traditional resources, and (3) design recommendations to enhance engagement and learning outcomes. PACE contributes to advancing cost-effective, scalable programming education.

CCS Concepts

• **Computing methodologies** → **Artificial intelligence; Machine learning**; • **Applied computing** → **Education**; • **Human-centered computing** → **Interaction design**.

Keywords

LLM, SLM, PACE, Python, Tutor, Learning, Tutoring, Education

ACM Reference Format:

Muhtasim Ibteda Shochcho, Mohammad Ashfaq Ur Rahman, Shadman Rohan, Ashraf Islam, Hasnain Heickal, AKM Mahbubur Rahman, M. Ashraf Amin, and Amin Ahsan Ali. 2025. Improving User Engagement and Learning Outcomes in LLM-Based Python Tutor: A Study of PACE. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*, April 26–May 01, 2025, Yokohama, Japan. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3706599.3720240>

1 Introduction

Programming has become a foundational skill in today's digital world. However, access to quality programming education remains a challenge due to factors such as a shortage of trained tutors, large class sizes, and limited instructor availability for guided problem-solving and providing personalized feedback. To address these challenges, Intelligent Tutoring System(ITS) systems have been developed.

Intelligent Tutoring Systems (ITS) have undergone substantial evolution over time, delivering personalized learning experiences across diverse disciplines [5, 23]. Traditional ITS systems utilized techniques such as rule-based reasoning, data mining, and Bayesian networks to evaluate learners and deliver adaptive guidance [23, 36]. The design principles guiding these early systems primarily focused on minimizing cognitive load, providing immediate feedback, and adjusting the size of training packages to suit learners' needs [24]. Over time, ITS systems progressed to integrate insights from cognitive science and educational research [24, 31, 33]. More recently, advancements in LLMs [4] have shown potential as a foundation for ITS [25], offering new avenues for personalization and interactivity.

We define the pedagogical use of LLMs as usage that focuses on tutoring instead of answering user queries. This may come in the form of different pedagogical strategies, such as scaffolding [38], analogies [9], metacognitive prompts [28] and personalized feedback [30]. While traditional LLMs like Llama 3.1, GPT-4o, etc, can support pedagogical interactions, they require carefully constructed prompts and state management to effectively implement instructional scaffolding. Moreover, access to these SoTA models

may not always be feasible due to technical or financial constraints, as training and running large models require significant computational resources[21], high energy consumption [34], and specialized expertise.

To address these limitations, small language models (SLMs) have emerged as a promising alternative [41] with some distinct advantages for this role: faster inference enables better real-time engagement, on-device deployment support [1] for classrooms with limited connectivity, and lower computational footprint make personalized tutoring economically viable at scale. However, these SLMs often lack the knowledge or linguistic capability of a large LLM and, therefore, require some task-specific adaptation [20].

To better understand learners' preference for SLMs, in this work, we try to address the following research questions (RQs):

- **RQ1:** How effectively can smaller language models (SLMs) address tutoring challenges in the context of programming?
- **RQ2:** To what extent do students trust AI-based programming tutors compared to traditional educational resources?
- **RQ3:** What design recommendations can be made to enhance user engagement and improve learning outcomes in LLM-based ITS for programming?

Building on the above questions, we introduce PACE (Python AI Companion for Enhanced Engagement), an intelligent SLM-based tutoring system designed to provide personalized, scalable, and adaptive support for Python programming education. To evaluate PACE, we conducted a study involving learners with different maturity levels. The results revealed some satisfaction with motivational feedback and clarity. Also identified some areas for enhancement, such as contextual retention and more tailored responses.

This paper is organized as follows: related work is presented in Section 2, system design in Section 3, methodology in Section 4, findings in Section 5, discussion in Section 6, and conclusion in Section 7.

2 Related Work

In this section, we discuss the pedagogical use of LLMs, followed by common challenges associated with virtual learning.

GPT-3 [4] pioneered LLM-based pedagogy, inspiring early research on educational applications [18, 29]. Studies have shown that LLM-based explanations can enhance learning, especially when students attempt problems before consulting them [5, 14, 16]. Personalized tutoring systems have also been developed incorporating student modeling and LLM-driven conversations, demonstrating the feasibility of tailored instruction [5, 27, 33]. To address concerns about LLMs deviating from desired pedagogical strategies, some researchers propose hybrid approaches that combine LLMs with pre-defined tutoring structures [33]. This method retains traditional pedagogical designs while leveraging LLM flexibility, potentially outperforming free-form LLM tutoring [26]. These advancements suggest a promising future for LLM-based tutoring systems, balancing AI capabilities with established educational practices.

Despite their high capability, concerns about LLMs' high computational demands, cost, and privacy have led researchers to explore smaller models. Studies show that well-trained SLMs can achieve impressive reasoning capabilities even with limited parameter count [20, 22]. Models like TinyLlama [40] and MiniCPM [13]

have demonstrated exceptional performance on downstream tasks through optimized training strategies, while Phi-2 [15], though significantly smaller, achieved strong results on reasoning benchmarks.

Building on these advancements, [19] provides empirical evidence that SLMs enhanced with Retrieval-Augmented Generation (RAG) can match or exceed popular LLMs like GPT-3.5 and GPT-4 in computing education contexts while addressing privacy concerns through local deployment. Specific implementations such as PyTutor [39], a ChatGPT-based intelligent tutoring system for Python education, have reported improved learner engagement and completion rates in programming tasks. However, our proposed system distinguishes itself by utilizing smaller fine-tuned LLMs, without any retrieval mechanism, optimized for domain-specific knowledge and tutoring.

Despite the promise of intelligent tutoring systems (ITS), several common challenges associated with virtual learning have been reported. Common challenges include designing effective learning activities [8], learners struggling to adapt to virtual environments [6], and addressing limitations of online tutoring, including student engagement and scalability [7].

3 System Design

This section describes the main components of the PACE tutorbot system, including model selection, dataset creation, model fine-tuning, and interface design. The design was adopted from the CLASS framework [33], using scaffolding and conversational data to build a functional tutoring system. Two datasets were generated to equip smaller models with the necessary skills for domain-specific tutoring.

3.1 Model Selection

To support a cost-effective solution, a smaller model, Gemma 2B from Google DeepMind [35], was selected for fine-tuning using LoRA (Low-Rank Adaptation) [37].

3.2 Dataset Design and Fine-Tuning

Two datasets were generated to fine-tune the Gemma 2B model. These datasets were generated using GPT-3.5 Turbo with prompts adapted from the CLASS framework [33]. The dataset generation prompt is in Appendix A, and examples are in Appendix B.

- **Scaffolding Dataset** contains Python problems broken down into smaller sub-problems, each including questions, answers, hints, and incorrect responses with feedback, facts, and solutions for the main problem. Skill levels were defined in the prompt using Bloom's Taxonomy: "lacking background knowledge" mapped to "Remember" and "Understand," "syntax errors" to "Analyze," and "logic implementation challenges" to "Apply" and "Create."
- **Conversational Dataset** was created to train the system to simulate interactive, tutor-like conversations. This dataset's characteristics are dialogues where scaffolding data was used in the prompt to ensure contextually relevant interactions and step-by-step feedback.

3.3 Interface Design

The user interface of our system, as shown in Figure 2a, is designed to be intuitive and engaging, prioritizing ease of use. Users can keep track of their "Chat Turns" graph within the dashboard, which shows their chat engagement with the system. Users can access their chat history by pressing the "Chat Turn Graph" Figure 2a; however, features like chapter progress tracking, learning streak tracking, and chapter progress are still under development and serve as non-functional placeholders. The chat interface, presented in Figure 2b, functions as a tutoring system, delivering real-time explanations and feedback. Users can interact with this chat tool by entering questions related to their learning experiences in the right block Figure 2b. Once the user types their question in the text box, just hit 'Enter' or click the 'Send' button to receive prompt answers.

4 Method

To address the research questions, we conducted a user study with 23 participants, selected through purposive sampling for diverse educational backgrounds. The study involved an interaction phase to explore the system and a feedback phase with a semi-structured questionnaire. Quantitative data assessed usability metrics, while qualitative responses underwent thematic analysis [3] to identify challenges and design recommendations.

4.1 Participants

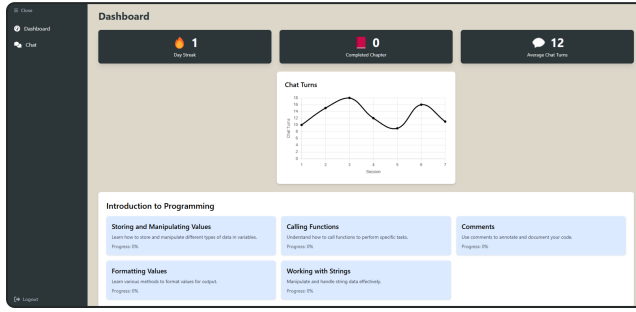
The study involved 23 participants selected via purposive sampling to ensure diversity in educational backgrounds and demographics. Participants included freshers (17.4%), juniors (8.7%), seniors (26.1%), and graduates (47.8%), with graduates considered programming experts. Ages ranged from 21 to 28 years (mean = 24.26, SD = 1.79), and the gender distribution was 2 females (8.7%) and 21 males (91.3%). Participants details are provided in Appendix 3.

4.2 Apparatus

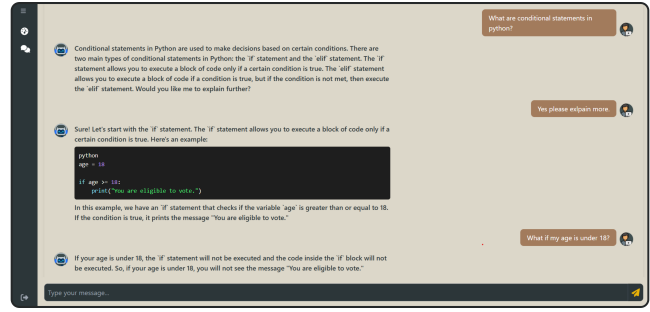
The PACE system, hosted on a local PC with an RTX 4090 GPU (24 GB VRAM), was accessed remotely via AnyDesk <https://anydesk.com/en>. Participants used the chat-based tutoring interface independently, with varied interaction times and query lengths, fully experiencing the system's capabilities.

4.3 Study Procedure

Participants started the study session by signing a consent form and providing demographic information. They were informed about the purpose of collecting chat interaction data and introduced to the PACE tutor bot's features through a demonstration of how it processes queries and generates responses. Participants then used the chat feature to explore PACE's capabilities by asking questions on topics such as basic arithmetic, conditional statements, loops, Python-specific functions, and complex programming challenges with chat durations ranging from 15 to 55 minutes (mean = 30.17, SD = 9.5) and chat turn counts ranging from 9 to 43 (mean = 18.17, SD = 7.48). After the interaction, participants completed a survey assessing the system's performance and usability, focusing on response quality metrics in Appendix C and usability given in Appendix C.6.



(a) Dashboard displaying chat counts and progress



(b) Student interacting with PACE chatbot

Figure 2: User Interface snapshots of PACE showcasing: (a) Students interacting with PACE and (b) The student progress dashboard.

The survey also included questions found in Appendix C.7 regarding trust in AI-generated content, as well as open-ended feedback on interface improvements, feature suggestions, and recommendations. This feedback was analyzed to identify actionable insights for enhancing the system. These survey questions were created in an informed way based on several brainstorming sessions and adapted from validated instruments like the System Usability Scale (SUS) and previous research on AI tutoring systems.

5 Findings

This section presents findings based on user feedback, organized around three research questions. We analyzed five key metrics to evaluate the effectiveness of PACE (RQ1). We also investigated user trust in AI-Tutor (RQ2) and examined qualitative feedback through thematic analysis [3] to identify gaps in the system and formulate design recommendations from users (RQ3).

5.1 Effectiveness of the PACE System

We evaluated the effectiveness of the PACE tutor bot by analyzing user responses based on five key metrics: Relevancy (R), Correctness (C), Completeness (Co), Motivation (M), and Clarity (Cl). Relevancy, Correctness, Motivation, and Completeness are drawn from [33], while Clarity was specifically defined to provide more accurate insights. Each metric was scored on a 5-point Likert scale (1 = Strongly Disagree to 5 = Strongly Agree) to assess distinct aspects of the system's performance. More about these metrics are discussed in Appendix C.

The results in Table 1 showed **Correctness** had the highest mean score (3.58, SD = 0.94), indicating the tutorbot's ability to provide accurate responses. **Clarity** followed with a mean score of 3.38 (SD = 0.93), suggesting responses were reasonably understandable. However, **Completeness** had the lowest score (2.92, SD = 0.85), highlighting a tendency for answers to lack detail or fully address user queries. The discrepancy between high Correctness and low Completeness highlights a reliance on brief responses, which may be suitable for specific clarifications but insufficient for in-depth learning.

This shortcoming appeared to impact other metrics, as **Relevancy** (3.29, SD = 0.95) and **Motivation** (3.11, SD = 0.97) also showed moderate performance, likely due to incomplete responses

Table 1: Performance Metrics for Chatbot Responses.

Metric	Mean	SD
Relevancy (R)	3.29	0.95
Correctness (C)	3.58	0.94
Completeness (Co)	2.92	0.85
Motivation (M)	3.11	0.97
Clarity (Cl)	3.38	0.93

reducing their overall impact. Additionally, the variability in **Motivation** (SD = 0.97) suggests inconsistent engagement, likely tied to the perceived relevance and depth of the tutorbot's answers. Addressing Completeness could enhance related metrics and provide a more balanced learning experience.

In addition to these metrics, the System Usability Scale (SUS) score revealed a usability score of 77.28, which corresponds to a grade of B+, placing PACE in the 80–84 percentile range [17]. This score indicates that the system falls within the acceptable category regarding usability [2].

5.2 Perceived Trust in the PACE System

The survey highlighted mixed perceptions of the PACE system. From the result in Table 2, Participants showed a willingness to continue using it (Q1: mean = 4.04, SD = 1.07) and to recommend it to others (Q2: mean = 3.96, SD = 1.19). However, trust in PACE was notably lower compared to teachers (Q3: mean = 2.78, SD = 1.28) and traditional resources (Q4: mean = 2.74, SD = 1.01), indicating areas for improvement in building reliability and authority. Participants highly valued tailored responses (Q5: mean = 4.26, SD = 0.86), emphasizing the need for adaptive, personalized feedback. Satisfaction with the system (Q6: mean = 3.04, SD = 0.98) was moderate, with negative correlations suggesting that more experienced and frequent ChatGPT users were less satisfied with PACE. This could be attributed to higher expectations or comparisons to advanced systems like ChatGPT, especially in response quality.

5.3 Design Recommendations

We thematically [3] analyzed open-ended survey responses to identify design recommendations for improving user engagement and learning outcomes in LLM-based programming tutors. Main themes

Table 2: Participant Ratings on Trust, Satisfaction, and Willingness to Use the PACE Tutorbot.

Question	Mean	SD
Q1. Willingness to continue using PACE	4.04	1.07
Q2. Recommend PACE to others	3.96	1.19
Q3. Trust in PACE compared to teachers	2.78	1.28
Q4. Trust in PACE compared to traditional resources	2.74	1.01
Q5. Preference for tailored responses	4.26	0.86
Q6. Satisfaction with PACE Tutorbot	3.04	0.98

are categorized as feature improvements, technical suggestions, user interface refinements, performance optimization, and general usability recommendations. Three researchers coded These themes collaboratively and augmented them with participant quotes to provide deeper context. A comprehensive breakdown of themes, subthemes, and codes is provided in Table 4. The themes, along with the major recommendations, are discussed below:

5.3.1 Feature Enhancement. Participants suggested several enhancements to improve the functionality and interactivity of the tutorbot. Voice Input was proposed by P13, who stated, “Having a voice input feature like ChatGPT would be nice.” The addition of Multiple Chat Sessions was emphasized by P15, who commented, “There is only one chat session/window; there should be more chat sessions so that we can interact on different topics on new chat like ChatGPT.” Code Output Verification was another key suggestion, with P17 noting, “It would be helpful if the tutorbot could show the output of the code it wrote, so learners can cross-check their outputs and improve their understanding.” P11 highlighted the importance of Short and Engaging Content, critiquing, “The chat was too lengthy. It should be more like examples and explanations and then should be exercise for instant testing.” These suggestions highlight the need for features that promote an interactive and learner-centric environment.

5.3.2 Technical Suggestions. Participants emphasized the need for improved technical capabilities. Larger models were recommended, with P16 stating, “Use a larger model because 3B parameter models are not meant for complex tasks like this.” Instruction-tuned models, such as StarCoder or Qwen-coder, were also suggested for their detailed responses. Regarding response quality, P16 noted, “Model responses were too short. Add depth by including related topics, like ternary search when explaining binary search.” The addition of Visual Language Models (VLMs) was proposed by P1: “Real-life VLMs should generate pictures of objects, buildings, or nature.” For personalized learning paths, P2 suggested enhancing the tutorbot with multi-modal support and integrated code execution. Code compilation was flagged as missing, with P9 stating, “Code compilation capability is missing.” P16 proposed a built-in Python IDE for editing and running code within the app. Lastly, for usability, P14 recommended allowing users to copy code snippets directly.

5.3.3 User Interface. Participants provided valuable feedback on the user interface. P3 highlighted inconsistent background colors, recommending a “modern and consistent palette.” A multiline input box was suggested by P3, who noted, “Multiline should be added.”

Improvements to loading animations were proposed, with P7 requesting a “loading dot is needed” and P23 adding, “A loading animation would enhance the experience.” Text alignment and clarity were emphasized, with P3 and P13 suggesting “step-by-step formatting and bullet points.” Code blocks were a concern, as P3 noted, “Code should appear in a proper block,” and P13 pointed out, “Sometimes code appears as raw text.” Font size and spacing improvements were also recommended by P3, and P5 suggested “Allow editing previous prompts.” P14 emphasized clarity, stating, “Start new points on separate lines.” While the simplicity of the UI was appreciated, these suggestions point to a need for a more polished and interactive design.

5.3.4 Performance & Accuracy. Several participants shared concerns about accuracy and efficiency. P6 stated, “System accuracy is low; responses are small and not well-defined.” P17 highlighted, “The bot struggles to understand some prompts.” On Handling Common Terms, P15 remarked, “When I say terms like ‘great,’ it acts like a student. Fix this issue.” P22 emphasized Correctness of Content, noting, “The chatbot didn’t provide code snippets and gave wrong answers.” P8 noted issues with Multiple Queries, stating, “The bot fails to understand if more than one question is asked together.” Tailored Answers were also suggested, as P8 explained, “The bot uses complex terms unsuitable for beginners.” Efficiency improvements were highlighted by P21, who observed, “PACE was very slow in responses and inefficient with typos.”

5.3.5 General Suggestions. Internet searched response was proposed by P6: “Give it Internet access and avoid using library functions.” Multilingual Support was also recommended, with P7 suggesting, “Maybe add a Bangla version.” For Beginner-Friendly Code, P6 emphasized, “It should write code in the easiest way without advanced syntax.” P13 proposed more detailed explanations: “A detailed or brief explanation for questions would be nice.” Flow Diagrams and Figures were suggested by P18: “Add figure generation for flow diagrams or processes.” Guided Learning Paths were recommended by P23, who noted, “Provide a path to follow for better flow.” Gamification was a recurring theme, with P11 stating, “Add features like leaderboards and quizzes.” P2 elaborated: “Include gamification, analytics dashboards, offline access, and integration with LMS or collaboration tools.”

6 Discussion

The findings of this study provide important insights into the usability and limitations of the PACE system as an AI-based tutor for programming education. Key observations, limitations, and potential improvements are discussed below.

6.1 Addressing the Diverse Needs of Learners

The observations highlight significant diversity in students’ learning strategies and preferences. Some students excel with quick explanations, others through problem-solving, while many benefit from coding examples. However, the current system does not adequately accommodate these diverse needs. For example, while PACE provides step-by-step guidance, it lacks integrated guided projects, implementation-based exercises, and multimodal options like visual aids or audio support. These gaps emphasize the need

for a more adaptive system that dynamically adjusts to individual learning styles, promoting better engagement and tailored educational experiences. Attention span poses a significant challenge for students, who often lose focus when outcomes are not achieved quickly or guidance is unclear. This reflects concerns about systems like ChatGPT, which prioritize quick solutions over deeper problem-solving skills. To address this, PACE should balance exploratory learning with efficiency and clarity. Incorporating scaffolded exercises, interactive content, and problem-solving prompts could foster deeper conceptual understanding and sustained attention.

6.2 Limitations and System Constraints

Despite its strengths, the PACE system has several limitations that hinder its effectiveness as a tutoring tool. One key issue is the occasional lack of accurate or comprehensive responses. As reflected in the performance metrics, Completeness scored the lowest among assessed categories, indicating the need for improved response depth and detail. Addressing this limitation requires integrating models capable of self-evaluating generated content for accuracy and relevance. Another major limitation is PACE cannot assess individual students' knowledge due to a lack of knowledge tracing, preventing tailored guidance or pinpointing difficulties. Its inability to connect with teachers or external sources further limits context-specific support, highlighting the need for adaptive learning and collaborative educator feedback tools. Additionally, the absence of motivational features, such as gamification, reduces user engagement over time. The absence of rewards or progress tracking reduces students' sense of achievement and hinders engagement. Addressing this could improve user retention and foster accomplishment. Our study focused on system usability and user trust in SLM-based tutoring without comparing it to LLM-based tutoring. We examined participant perceptions and engagement patterns for future improvements. However, with nearly half of the participants being graduate programming experts, there is a selection bias that limits the applicability of our findings to beginner programmers. Future research should aim for a more balanced participant pool.

6.3 Future Directions

Future improvements to PACE should focus on building more adaptive systems, such as progress-tracking and leaderboard, as research shows gamification boosts motivation and sustained learning [32]. Adding dynamic assessments like quizzes and guided exercises can help track progress and reinforce concepts [11]. Additionally, implementing diagnostic features can personalize learning paths by identifying gaps in knowledge [10]. Lastly, incorporating interactive coding challenges within the chat—where users debug, complete, or analyze snippets—bridges theory and application, a proven strategy in programming education [12]. These updates can make PACE more engaging, accurate, and effective for Python programming education.

7 Conclusion

The PACE system demonstrated its effectiveness in delivering accurate and tailored responses for programming education in Python, achieving positive usability ratings. However, users highlighted a few areas for improvement, including better context retention,

enhanced interactivity (e.g., voice input, multi-session support), and integrated code verification. Trust in PACE was lower compared to traditional resources, underscoring the need for increased reliability and response quality. Key recommendations include leveraging larger, instruction-tuned models, improving interface design, and addressing performance issues like response speed and accuracy. By addressing these gaps, PACE can better support cost-effective, scalable programming education and foster greater user trust and engagement.

Acknowledgments

We thank our participants for their time and contribution to this research. This work was supported by a sponsored research grant from Independent University, Bangladesh (IUB).

References

- [1] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, et al. 2024. Phi-3 technical report: A highly capable language model locally on your phone.
- [2] Aaron Bangor, Philip Kortum, and James Miller. 2009. Determining what individual SUS scores mean: Adding an adjective rating scale. *Journal of usability studies* 4, 3 (2009), 114–123.
- [3] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative research in psychology* 3, 2 (2006), 77–101.
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [5] Yulin Chen, Ning Ding, Hai-Tao Zheng, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. 2024. Empowering Private Tutoring by Chaining Large Language Models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management* (Boise, ID, USA) (CIKM '24). Association for Computing Machinery, New York, NY, USA, 354–364. <https://doi.org/10.1145/3627673.3679665>
- [6] Baris Cukurbasi and Mubin Kiyici. 2021. Instructional design and instructional effectiveness in virtual classrooms: Research trends and challenges: Instructional Design and Instructional Effectiveness in Virtual Classroom. *Australasian Journal of Educational Technology* 37, 6 (2021), 156–174.
- [7] Marija Djokic-Petrovic, David Pritchard, Milos Ivanovic, and Vladimir Cvjetkovic. 2016. IMI Python: Upgraded CS circles web-based python course. *Computer Applications in Engineering Education* 24, 3 (2016), 464–480.
- [8] Geela Venise Firmalo Fabic, Antonija Mitrovic, and Kourosh Neshatian. 2018. Investigating the effects of learning activities in a mobile Python tutor for targeting multiple coding skills. *Research and practice in technology enhanced learning* 13 (2018), 1–24.
- [9] Dedre Gentner. 1983. Structure-mapping: A theoretical framework for analogy. *Cognitive science* 7, 2 (1983), 155–170.
- [10] Ilie Gligorea, Marius Cioca, Romana Oancea, Andra-Teodora Gorski, Hortensia Gorski, and Paul Tudorache. 2023. Adaptive learning using artificial intelligence in e-learning: a literature review. *Education Sciences* 13, 12 (2023), 1216.
- [11] Elena L Grigorenko and Robert J Sternberg. 1998. Dynamic testing. *Psychological bulletin* 124, 1 (1998), 75.
- [12] Mark Guzdial, Cindy Hmelo, Roland Hübscher, Kris Nagel, Wendy Newstetter, Sadhana Puntambekar, Amnon Shabo, Jennifer Turns, and Janet L. Kolodner. 1997. Integrating and guiding collaboration: lessons learned in computer-supported collaborative learning research at Georgia Tech. In *Proceedings of the 2nd International Conference on Computer Support for Collaborative Learning (CSCL '97)*. International Society of the Learning Sciences, Toronto, Ontario, Canada, 95–105.
- [13] Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. 2024. Minicpm: Unveiling the potential of small language models with scalable training strategies.
- [14] Wenyue Hua, Lei Li, Shuyuan Xu, Li Chen, and Yongfeng Zhang. 2023. Tutorial on Large Language Models for Recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems* (Singapore, Singapore) (RecSys '23). Association for Computing Machinery, New York, NY, USA, 1281–1283. <https://doi.org/10.1145/3604915.3609494>
- [15] Mojan Javaheripi, Sébastien Bubeck, Marah Abdin, Jyoti Aneja, Sébastien Bubeck, Caio César Teodoro Mendes, Weizhu Chen, Allie Del Giorno, Ronen Eldan, Sivakanth Gopi, et al. 2023. Phi-2: The surprising power of small language models. *Microsoft Research Blog* 1, 3 (2023), 3.

- [16] Harshit Kumar, David M. Rothschild, Daniel G. Goldstein, and Jake Hofman. 2023. Math Education with Large Language Models: Peril or Promise? *SSRN Electronic Journal* N/A (2023), N/A.
- [17] James R Lewis. 2018. The system usability scale: past, present, and future. *International Journal of Human–Computer Interaction* 34, 7 (2018), 577–590.
- [18] Zhenwen Liang, Wenhao Yu, Tanmay Rajpurohit, Peter Clark, Xiangliang Zhang, and Ashwin Kalyan. 2023. Let GPT be a Math Tutor: Teaching Math Word Problem Solvers with Customized Exercise Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 14384–14396. <https://doi.org/10.18653/v1/2023.emnlp-main.889>
- [19] Suqing Liu, Zezhu Yu, Feiran Huang, Yousef Bulbulia, Andreas Bergen, and Michael Liut. 2024. Can Small Language Models With Retrieval-Augmented Generation Replace Large Language Models When Learning Computer Science?. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1* (Milan, Italy) (*ITICSE 2024*). Association for Computing Machinery, New York, NY, USA, 388–393. <https://doi.org/10.1145/3649217.3653554>
- [20] Lucie Charlotte Magister, Jonathan Mallinson, Jakub Adamek, Eric Malmi, and Aliaksei Severyn. 2022. Teaching small language models to reason.
- [21] Ben Mann, N Ryder, M Subbiah, J Kaplan, P Dhariwal, A Neelakantan, P Shyam, G Sastry, A Askell, S Agarwal, et al. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* 1 (2020), 3.
- [22] Arindam Mitra, Luciano Del Corro, Shweti Mahajan, Andres Codas, Clarisse Simoes, Sahaj Agarwal, Xuxi Chen, Anastasia Razdaibiedina, Erik Jones, Kriti Aggarwal, et al. 2023. Orca 2: Teaching small language models how to reason.
- [23] Elham Mousavinasab, Nahid Zarifsanaiy, S. R. N. Kalthori, M. Rakhshan, L. Keikha, and M. Saeedi. 2018. Intelligent tutoring systems: a systematic review of characteristics, applications, and evaluation methods. *Interactive Learning Environments* 29 (2018), 142 – 163. <https://doi.org/10.1080/10494820.2018.1558257>
- [24] Cristina Niculescu. 2016. Intelligent tutoring systems-trends on design, development and deployment. *Conference proceedings of eLearning and Software for Education* 12, 03 (2016), 280–285.
- [25] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- [26] Sankalan Pal Chowdhury, Vilém Zouhar, and Mrinmaya Sachan. 2024. AutoTutor meets Large Language Models: A Language Model Tutor with Rich Pedagogy and Guardrails. In *Proceedings of the Eleventh ACM Conference on Learning @ Scale* (Atlanta, GA, USA) (*L@S '24*). Association for Computing Machinery, New York, NY, USA, 5–15. <https://doi.org/10.1145/3657604.3662041>
- [27] Minju Park, Sojung Kim, Seunghyun Lee, Soonwoo Kwon, and Kyuseok Kim. 2024. Empowering Personalized Learning through a Conversation-based Tutoring System with Student Modeling. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (*CHI EA '24*). Association for Computing Machinery, New York, NY, USA, Article 123, 10 pages. <https://doi.org/10.1145/3613905.3651122>
- [28] Paul R Pintrich. 2002. The role of metacognitive knowledge in learning, teaching, and assessing. *Theory into practice* 41, 4 (2002), 219–225.
- [29] Joel Manuel Prieto-Andreu and Antonio Labisa-Palmeira. 2024. Quick Review of Pedagogical Experiences Using GPT-3 in Education. *Journal of Technology and Science Education* 14, 2 (2024), 633–647.
- [30] Valerie J Shute. 2008. Focus on formative feedback. *Review of educational research* 78, 1 (2008), 153–189.
- [31] Enrique Sierra, R. G. Martínez, Z. Cataldi, P. Britos, and Alejandro Hossian. 2006. Towards a Methodology for the Design of Intelligent Tutoring Systems. *Research on computing science* 20 (2006), 181–190.
- [32] Rodrigo Smiderle, Sandro José Rigo, Leonardo B Marques, Jorge Arthur Pechanha de Miranda Coelho, and Patricia A Jaques. 2020. The impact of gamification on students' learning, engagement and behavior based on their personality traits. *Smart Learning Environments* 7, 1 (2020), 3.
- [33] Shashank Sonkar, Naiming Liu, Debshila Mallick, and Richard Baraniuk. 2023. CLASS: A Design Framework for Building Intelligent Tutoring Systems Based on Learning Science principles. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 1941–1961. <https://doi.org/10.18653/v1/2023.findings-emnlp.130>
- [34] Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2020. Energy and Policy Considerations for Modern Deep Learning Research. *Proceedings of the AAAI Conference on Artificial Intelligence* 34, 09 (Apr. 2020), 13693–13696. <https://doi.org/10.1609/aaai.v34i09.7123>
- [35] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. 2024. Gemma: Open models based on gemini research and technology.
- [36] Elena Verdú, Luisa M Regueras, María Jesús Verdú, Juan Pablo de Castro, Dan Kohen-Vacs, Eran Gal, and Michaela Ronen. 2014. Intelligent tutoring interface for technology enhanced learning in a course of computer network design. In *2014 IEEE Frontiers in Education Conference (FIE) Proceedings*. IEEE, IEEE, Madrid, Spain, 1–7.
- [37] Xi Wang, Laurence Aitchison, and Maja Rudolph. 2023. LoRA ensembles for large language model fine-tuning. <https://doi.org/10.48550/arXiv.2310.00035>
- [38] David Wood, Jerome S Bruner, and Gail Ross. 1976. The role of tutoring in problem solving. *Journal of child psychology and psychiatry* 17, 2 (1976), 89–100.
- [39] Albert CM Yang, Ji-Yang Lin, Cheng-Yan Lin, and Hiroaki Ogata. 2024. Enhancing python learning with PyTutor: Efficacy of a ChatGPT-Based intelligent tutoring system in programming education. *Computers and Education: Artificial Intelligence* 7 (2024), 100309.
- [40] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. Tinyllama: An open-source small language model.
- [41] Qin Zhang, Ziqi Liu, Shirui Pan, and Can Wang. 2025. The Rise of Small Language Models. *IEEE Intelligent Systems* 40 (2025), 30–37. <https://api.semanticscholar.org/CorpusID:276510647>

A Data Generation Prompts

A.1 Scaffolding Data Generation Prompt

Generate a hard, challenging, and unique Python problem which can be broken down into subproblems for the following section on {section_name}, whose learning objective is: {section_learning_objective}, and skill level is {skill_level}.

For the generated main problem for this learning objective, also output the following:

- 1) Facts necessary to answer it,
- 2) Subproblems that the main problem can be broken down into,
- 3) The final answer.

For each subproblem, generate A hint, One incorrect student response to the subproblem, and Corresponding feedback to the student.

Put all the output in the following JSON structure

```
{
  "Problem": "...",
  "SubProblems": [
    {
      "Question": "...",
      "Answer": "...",
      "Hint": "...",
      "Incorrect Response": "...",
      "Feedback": "..."
    }
  ],
  "Facts": [
    "...",
    "..."
  ],
  "Solution": "..."
}
```

Only provide this JSON result in the output; no other irrelevant messages are required.

A.2 Conversational Data Generation Prompt

Your goal is to create a mock conversation between Student and a Python Tutorbot, an AI-powered chatbot designed to help Student's with a question regarding python programming questions, debugging and error identification. Student will ask a question based on their skill level .

Context: {problems}

Student Skill Level: {skill_level}

Now simulate the conversation in this manner:

```
"Student": "...",
"Thoughts of Tutorbot": "...
"Evaluation of Student Response": "...
"Action Based on Evaluation": "...
"Subproblem State": "...
"Subproblem": "...
"Tutorbot": "Let's break the problem into
subproblems and tackle the subproblems one by
one. Let's begin with the first subproblem
...",
```

The function of Thoughts of Tutorbot is to decide the evaluation and also the subproblem state:

- a) Evaluating Incorrect Responses
- b) Evaluating Correct Responses
- c) Evaluating Partially Correct Responses
- d) Evaluating Ambiguous or Unclear or Short Responses
- e) Redirecting Off-topic Responses
- f) Responding to Student Inquiries
- g) N/A

Tutorbot Actions Based on the Evaluation:

If "a" is the evaluation, then:

- 1) Promptly notify the student about the mistake, Provide constructive feedback to pinpoint the errors, Offer helpful hints
- 2) Step in to provide a solution if the student is unable to answer even after multiple attempts.

If "b" is the evaluation, then:

- 3) Confirm the correct answer. Check for completeness for the answer to the subproblem. If

solution is incomplete, notify the student to complete the solution.

If "c" is the evaluation, then:

- 4) Acknowledge the accurate parts, Promptly notify the student about the mistake, Provide constructive feedback to pinpoint the errors, Offer helpful hints

- 5) Step in to provide a solution if the student is unable to answer even after multiple

attempts.

If "d" is the evaluation, then:

- 6) Actively seek clarification through relevant follow-up questions. Request the student to provide more specific information.

If "e" is the evaluation, then:

- 7) Skillfully redirect the student's attention to the subject matter. Provide guidance on how to approach the question appropriately.

If "f" is the evaluation, then:

- 8) If student asks for a hint, provide a hint for the current subproblem.
- 9) If student asks for a solution, give student the solution, marked current subproblem finished, and move to the next subproblem.
- 10) If student asks to move to previous subproblem, marked current subproblem finished, and move to the previous subproblem.

- 11) If none apply, prioritize addressing the inquiry. Offer relevant support and guidance to meet the student's specific needs.

If "g" is the evaluation, then:

- 12) N/A

Function of Subproblem State is to guide through subproblems:

- w) N/A

- x) One of the subproblems is currently being solved
- y) Subproblem finished, moving to next subproblem that is not finished
- z) Subproblem finished, no next subproblem, problem finished

Now, let's begin. Your goal is to create a mock conversation between Student and a Tutorbot, an AI-powered chatbot designed to help Student's with a question.

Please create a mock conversation now. Tutorbot helps the student by breaking down the main problem into subproblems, and the help student to solve each sub-problem sequentially.

Tutorbot only provide hints.

Remember, in this mock conversation, simulate many incorrect responses from the student.

Use the following json format:

Put all the output in the following JSON structure

```
[{
  "Student": "...",
  "Thoughts of Tutorbot": "...
  "Evaluation of Student Response": "a,b,c,d,e,f,g"
  "Action Based on Evaluation":
    "1,2,3,4,5,6,7,8,9,10,11,12"
  "Subproblem State": "w,x,y,z"
  "Subproblem": "...
  "Tutorbot": "...,
}],
Repeat above N times.
]
```

- Don't give any other messages except this json.
And don't repeat similar conversation always.
Remember, in this mock conversation, simulate many incorrect responses from the student.

B Dataset Examples

B.1 Scaffolding Data

```
{
  "Problem": "Write a Python program to print all
    prime numbers between 1 and 100.",
  "SubProblems": [
    {
      "Question": "How do we identify whether a
        number is prime or not?",
      "Answer": "We can check if a number has any
        divisors other than 1 and itself. If it
        does, then it's not a prime number.",
      "Hint": "Think about what makes a number
        prime...",
      "Incorrect Response": "A number is prime if
        it has more than two digits.",
      "Feedback": "That's incorrect! A prime
        number can have any number of digits, as
        long as it only has divisors 1 and
        itself."
    },
    {
      "Question": "How do we implement the loop to
        check for prime numbers?",
      "Answer": "We can use a `for` loop to
        iterate over all numbers from 2 to
        100.",
      "Hint": "Think about how you would write
        this loop...",
      "Incorrect Response": "We should use a `
        while` loop and start from 1.",
      "Feedback": "That's close, but not quite
        correct. We can indeed use a `for` loop
        here."
    },
    {
      "Question": "How do we check if a number is
        prime inside the loop?",
      "Answer": "We can use the divisibility rule
        to see if the current number has any
        divisors other than 1 and itself.",
      "Hint": "Think about how you would implement
        this check...",
      "Incorrect Response": "We should just print
        all numbers that are not divisible by
        themselves!",
      "Feedback": "That's close, but not quite
        correct. We need to be more specific
        about what makes a number prime."
    }
  ]
}
```

```
],
  "Facts": [
    "The smallest prime number is 2.",
    "Prime numbers only have divisors 1 and
      themselves."
  ],
  "Solution": "def print_primes(n): for i in range
    (2, n+1): is_prime = True for j in range(2,
    int(i**0.5) + 1): if i % j == 0: is_prime =
    False break if is_prime: print(i)"
}
```

B.2 Conversational Data

```
[{
  "Student": "I have a list of integers and I want
    to find the maximum sum of a contiguous
    sublist of odd numbers.",
  "Thoughts of Tutorbot": "Let's break down the
    problem into subproblems. We need to first
    find the maximum sum of a contiguous sublist
    , then check if all elements in that sublist
    are odd.",
  "Evaluation of Student Response": "f",
  "Action Based on Evaluation": "8",
  "Subproblem State": "w",
  "Subproblem": "How to find the maximum sum of a
    contiguous sublist?",
  "Tutorbot": "To find the maximum sum of a
    contiguous sublist, you can use Kadane's
    algorithm. Can you tell me how it works?"
},
{
  "Student": "I'll just sort the list and take the
    last element.",
  "Thoughts of Tutorbot": "Sorting the list does
    not guarantee finding the correct result.
    Let's think about how we can efficiently
    find the maximum sum of a contiguous sublist
    .",
  "Evaluation of Student Response": "d",
  "Action Based on Evaluation": "6",
  "Subproblem State": "w",
  "Subproblem": "How to find the maximum sum of a
    contiguous sublist?",
  "Tutorbot": "Can you tell me what's the main
    idea behind Kadane's algorithm? What are the
    key steps involved?"
},
{
  "Student": "I'm not sure, can you give me a hint
    ?",
  "Thoughts of Tutorbot": "Kadane's algorithm
    involves finding the maximum sum of a
    contiguous sublist by scanning the list from
    left to right and keeping track of the
    maximum sum seen so far.",
}
```

```

    "Evaluation of Student Response": "f",
    "Action Based on Evaluation": "8",
    "Subproblem State": "y",
    "Subproblem": "How to find the maximum sum of a
        contiguous sublist?",
    "Tutorbot": "That's correct! Now, let's move on
        to the next subproblem: How can we check if
        all elements in that sublist are odd?",
},
{
    "Student": "I'll just check if each element is
        odd by looking at its last digit.",
    "Thoughts of Tutorbot": "Checking if an element
        is odd by looking at its last digit does not
        work. We need to use the modulus operator
        (%) to determine whether a number is odd.",
    "Evaluation of Student Response": "d",
    "Action Based on Evaluation": "6",
    "Subproblem State": "y",
    "Subproblem": "How can we check if all elements
        in that sublist are odd?",
    "Tutorbot": "Can you tell me why checking the
        last digit is not sufficient? What's a
        better way to determine whether an element
        is odd?",
},
{
    "Student": "I'm still not sure, can I move on to
        the next subproblem?",
    "Thoughts of Tutorbot": "Sure! Now that we have
        found the maximum sum of a contiguous
        sublist and checked if all elements in that
        sublist are odd, let's combine these two
        ideas.",
    "Evaluation of Student Response": "e",
    "Action Based on Evaluation": "7",
    "Subproblem State": "z",
    "Subproblem": "None",
    "Tutorbot": "That's correct! Now we have found
        the maximum sum of a contiguous sublist of
        odd numbers."
}]

```

C User Evaluation Questionnaire

To evaluate the effectiveness and usability of PACE, participants completed a user survey that assessed the system across five metrics: Relevancy, Correctness, Completeness, Motivation, and Clarity. Below is an explanation of each metric, followed by the corresponding questions used in the survey.

C.1 Relevancy

This metric assesses how well the chatbot's responses align with the user's programming needs and expectations. It evaluates whether the chatbot provides contextually appropriate and meaningful suggestions, examples, and solutions.

- How relevant were the chatbot's suggestions for your programming challenges?
- How well did the chatbot's responses address the topics you were asking about?
- How appropriate was the chatbot for providing examples that fit your learning needs?
- How closely did the chatbot's responses align with your expectations for Python programming help?
- How applicable were the chatbot's answers to real-world programming situations?

C.2 Correctness

This metric evaluates the accuracy of the chatbot's responses, including syntax, definitions, and solutions. It measures whether the chatbot consistently provides correct and reliable information.

- Did the chatbot provide the correct syntax for the programming examples given?
- How accurately did the chatbot explain concepts related to Python programming?
- How often did the chatbot's solutions lead to correct outputs when tested?
- Did the chatbot provide the correct definitions for programming terms?
- How effectively did the chatbot correct your misconceptions about Python programming?

C.3 Completeness

Completeness measures whether the chatbot provided thorough and detailed responses. This includes offering all necessary steps, relevant examples, and comprehensive explanations to address user queries.

- Did the chatbot provide all the necessary steps to solve your programming issue?
- Did the chatbot include relevant examples when addressing your queries?
- How complete were the chatbot's explanations regarding Python programming concepts?
- How thoroughly did the chatbot cover the topic when responding to your questions?
- Did the chatbot recommend related topics for a deeper understanding of Python programming?

C.4 Motivation

Motivation assesses the extent to which the chatbot inspired users to continue learning and exploring programming. It examines whether interactions with the chatbot built confidence, sparked interest, and encouraged further practice.

- How inspired were you to continue coding after interacting with the chatbot?
- Did the chatbot encourage you to tackle more complex programming tasks?
- How motivated were you to practice Python programming after receiving help from the chatbot?
- Did the feedback from the chatbot make you feel more confident in your programming abilities?

- How much did the chatbot’s responses spark your interest in learning more about programming?

C.5 Clarity

Clarity evaluates the chatbot’s ability to communicate effectively, ensuring that instructions, explanations, and examples are easy to understand and follow.

- How clear were the chatbot’s instructions for fixing your coding errors?
- Did the chatbot explain the reasoning behind the solutions clearly?
- How easy was following the chatbot’s guidance when working through programming problems?
- Were the terms used by the chatbot always easy to understand?
- How straightforward were the chatbot’s examples in illustrating programming concepts?

C.6 SUS Questionnaire

The System Usability Scale (SUS) was used to assess the overall usability of the PACE system. Participants rated their agreement with the following statements on a 5-point Likert scale (1 = Strongly Disagree to 5 = Strongly Agree):

- I think that I would like to use this system frequently.
- I found the system unnecessarily complex.
- I thought the system was easy to use.
- I think that I would need the support of a technical person to be able to use this system.
- I found the various functions in this system were well integrated.
- I thought there was too much inconsistency in this system.
- I would imagine that most people would learn to use this system very quickly.
- I found the system very cumbersome to use.
- I felt very confident using the system.
- I needed to learn a lot of things before I could get going with this system.

C.7 Additional Usability Questions

To gather further insights into user experiences, participants answered additional usability and trust-related questions:

- How experienced are you with using AI tools (e.g., ChatGPT, Claude.ai)? (Not at all experienced to Extremely experienced)
- How long have you been using systems like ChatGPT or Claude.ai? (Less than 1 month to More than 2 years)
- How frequently do you use AI tools such as ChatGPT or Claude? (Rarely to Daily)
- How much do you trust AI-generated content? (Not at all to Completely)
- Do you prefer using the ChatGPT-like system on mobile or desktop devices? (Mobile/Desktop)
- What methods or resources do you use to learn Python? (Books, YouTube Videos, Tutors, Blogs, Documentation)

- How willing are you to continue using this type of Intelligent Tutor Systems (ITS) in the future? (Not willing at all to Extremely willing)
- Would you recommend the PACE to your friends, peers, or juniors who are learning to code? (Definitely not to Definitely yes)
- How do you rate your trust in PACE compared to teachers? (Much lower than teachers to Much higher than teachers)
- How do you rate your trust in PACE compared to traditional resources such as books and online resources? (Much lower to Much higher)
- Would you prefer tailored responses or guidance based on your previous interactions with the system? (Not at all to Very Much)
- On a scale from 1 to 5, how satisfied are you with your experience using our PACE tutorbot? (1 = Very Unsatisfied to 5 = Very Satisfied)
- What are your thoughts on the user interface? (Open-ended response)
- What additional features or integrations do you believe would enhance the PACE Tutorbot system? (Open-ended response)
- Please share any additional thoughts or suggestions you have regarding your experience with the PACE Tutorbot system. (Open-ended response)

D Characteristics of the participants

Table 3 shows participants’ demographic data from the survey.

Table 3: Demographic Data of Participants

Participant No.	Education Level	Age	Gender
1	Senior	25	M
2	Senior	24	M
3	Senior	24	M
4	Senior	24	M
5	Senior	24	M
6	Junior	23	M
7	Junior	23	M
8	Fresher	22	M
9	BSc Graduated	28	M
10	BSc Graduated	28	F
11	Fresher	22	M
12	Fresher	22	F
13	BSc Graduated	25	M
14	BSc Graduated	28	M
15	Senior	25	M
16	BSc Graduated	26	M
17	BSc Graduated	26	M
18	BSc Graduated	24	M
19	BSc Graduated	25	M
20	BSc Graduated	26	M
21	BSc Graduated	27	M
22	Fresher	21	M
23	BSc Graduated	24	M

E Thematic Analysis of Participant Feedback

The Table 4 presents a consolidated thematic analysis of participant feedback for improving the PACE tutorbot system. It categorizes suggestions into themes such as Feature Enhancements, Technical Suggestions, User Interface, Performance & Accuracy, and General Suggestions. Each theme is further divided into subthemes, with specific actionable codes identified for enhancing the system’s usability, trust, and engagement.

Table 4: Thematic Analysis for Improving PACE System

Feature Enhancements	
Subtheme	Codes
Voice Input	Add voice input functionality.
Multiple Chat Sessions	Enable multiple chat windows/sessions for different topics.
Code Output Verification	Display code outputs for learners to validate results.
Short and Engaging Content	Provide concise examples and instant exercises.
Technical Suggestions	
Subtheme	Codes
Larger Models	Use larger models for handling complex tasks efficiently.
Instruction-Tuned Models	Integrate models like StarCoder for better coding responses.
Response Quality	Diversify responses and introduce relevant related topics.
Visual Language Models (VLMs)	Support image generation for visual examples.
Personalized Learning Paths	Implement tailored, interactive learning paths.
Code Compilation	Enable direct code compilation within the platform.
Code Snippet Copying	Simplify copying code snippets with a one-click option.
User Interface	
Subtheme	Codes
Background and Colors	Ensure consistent background color and a modern palette.
Multiline Input Box	Allow multiline input for longer queries.
Edit Previous Prompts	Add functionality to edit previously entered prompts.
Loading Animation	Show a loading animation while processing answers.
Text Alignment and Clarity	Improve alignment, indentation, and readability of responses.
Code Blocks	Display code outputs in properly formatted code blocks.
Font and Spacing	Use larger font and improved spacing for better readability.
Performance & Accuracy	
Subtheme	Codes
Accuracy	Improve system accuracy and clarity of responses.
Handling Common Terms	Enhance understanding of simple terms and phrases.
Correctness of Content	Ensure correct responses and accurate code snippets.
Multiple Queries	Handle multiple questions asked together effectively.
Tailored Answers	Provide responses tailored to users’ expertise level.
General Suggestions	
Subtheme	Codes
Internet Search	Enable queries to find real-time information.
Multilingual Support	Add support for multiple languages, e.g., Bangla.
Beginner-Friendly Code	Avoid advanced unless requested.
Explanation Length	Let users choose between detailed or brief explanations.
Flow Diagrams and Figures	Generate flow diagrams for better visualization.
Guided Learning Paths	Add guided paths for structured and consistent learning.
Gamification	Include quizzes, leaderboards, and point systems.